



HiDPU: A DPU-Oriented Hybrid Indexing Scheme for Disaggregated Storage Systems

Wenbin Zhu, Zhaoyan Shen, and Qian Wei, *Shandong University*;
Renhai Chen, *Tianjin University and Huawei Technologies Co., Ltd*; Xin Yao, *Huawei Technologies Co., Ltd*; Dongxiao Yu, *Shandong University*; Zili Shao, *The Chinese University of Hong Kong*

<https://www.usenix.org/conference/fast25/presentation/zhu>

This paper is included in the Proceedings of the
23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings
of the 23rd USENIX Conference on
File and Storage Technologies
is sponsored by



HiDPU: A DPU-Oriented Hybrid Indexing Scheme for Disaggregated Storage Systems

Wenbin Zhu¹, Zhaoyan Shen^{1,*}, Qian Wei¹, Renhai Chen^{2,3,*}, Xin Yao³, Dongxiao Yu¹, and Zili Shao⁴

¹Shandong University

²Tianjin University

³Huawei Technologies Co., Ltd

⁴The Chinese University of Hong Kong

Abstract

Data Processing Units (DPUs) have been deployed in disaggregated storage systems to accelerate data transmission. However, in this paper, we observe that during data access in disaggregated storage, the address translation process incurs significant CPU computation overhead and leads to high system latency. Additionally, in large-scale storage systems, the address indexing structures also consume substantial memory space, incurring high costs.

To address these challenges, we propose HiDPU, a DPU-oriented hybrid indexing scheme optimized for disaggregated storage systems. Our solution introduces a multi-level indexing structure to alleviate the limitations of DPU memory resources, constrained computational power, and the high DPU-host interaction overhead. Mapping entries for the storage space are divided into different kinds of segments (i.e., accurate, PTHash, and LPTHash) to leverage address continuity. A layered learned index is constructed across these segments to enhance memory efficiency. To further reduce DPU-host interactions, small upper-layer indexes and frequently accessed metadata are maintained on the DPU, limiting interactions to a single instance. HiDPU also implements a two-phase asynchronous index update strategy to ensure index consistency between the DPU and host memory, while minimizing performance overhead. Experimental results on Huawei's Hi1823 DPU demonstrate that HiDPU achieves up to 92% memory savings and improves query performance by up to $6.3\times$ compared to existing solutions.

1 Introduction

Disaggregated storage systems decouple computation from storage, enabling independent scaling of resources and promoting more efficient resource sharing and data management [30, 46, 55, 68]. In this architecture, applications run on compute servers, while storage servers manage storage

resources (e.g. flash arrays) and present a unified large logical address space to the compute servers via a storage controller [21, 22, 67]. Benefiting from its increased elasticity, durability, and resource utilization, this architecture has been widely deployed in data centers.

However, in disaggregated storage systems, I/O requests experience longer transmission paths, particularly due to a more complex software stack, leading to higher latency. This issue becomes more pronounced with the growing demands for big data analytics and real-time processing [56, 61]. To address this, some solutions propose moving I/O operations into user space, eliminating OS kernel/user space memory copies, and reducing context switch overhead, such as through Remote Direct Memory Access (RDMA) [34] and the Storage Performance Development Kit (SPDK) [1, 40]. Additionally, several studies have leveraged emerging networking hardware, including Data Processing Units (DPUs) [9, 69], to streamline I/O processes and enhance efficiency.

DPUs combine dedicated networking and storage interfaces with programmable computing cores and hardware accelerators [28]. They are positioned on the PCIe system bus in the storage server and can have direct access to flash arrays. When a request arrives at the DPU through its network interface, it can directly read the corresponding data from the flash arrays on the back end and return the result, as illustrated in Figure 1a. This allows for the removal of the additional memory copy from the data path in the storage server, significantly streamlining data processing. However, reliance on CPU processing and the accompanying software overhead continue to constrain performance scaling.

In practice, storage server resources are typically managed by a host storage controller (represented by the CPU in Figure 1a), which exposes a unified logical address space to the computing server. The corresponding large logical-to-physical address mapping information is stored in the host controller's memory. As shown in the figure, before accessing data, the DPU must first retrieve the physical address by interacting with the CPU to query the index stored in host memory, which incurs substantial latency [2,

*Zhaoyan Shen (shenzhaoyan@sdu.edu.cn) and Renhai Chen (renhai.chen@tju.edu.cn) are the co-corresponding authors of this paper.

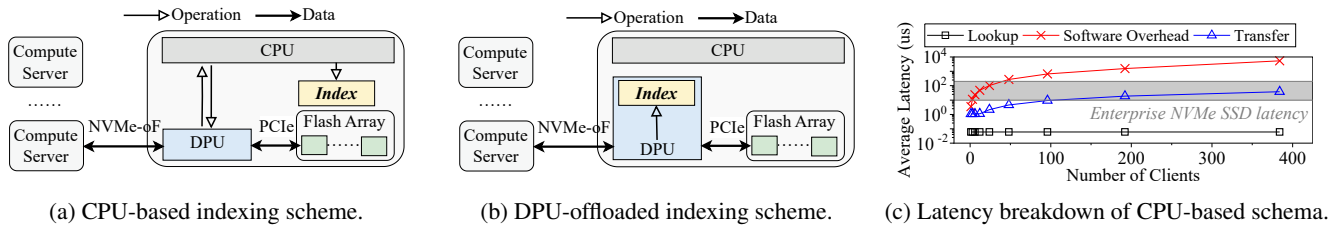


Figure 1: Data query process in DPU-assist disaggregated storage systems.

38]. Under heavy parallel I/O workloads, this CPU-based indexing process becomes a critical system bottleneck. This is problematic in disaggregated storage, where the CPU is already burdened with tasks such as space management and data replication [21].

We conducted preliminary experiments to measure the execution time of the indexing process as the number of requesting clients increased, with the results presented in Figure 1c. In this experiment, the DPU directly interacted with the CPU through PCIe and the DPU driver to avoid the serialization/deserialization and network protocol stack involved with Remote Procedure Call (RPC) [50]. However, the results demonstrate that as the number of clients increases, the overall latency also grows, rising from 4 μ s to over 5000 μ s and subsequently surpassing the latency of enterprise NVMe SSDs (10–200 μ s [19, 20]). Apart from the essential **Lookup** of the index and **Transfer** of the physical address through PCIe, **Software Overhead**—including CPU-side buffer allocation, thread scheduling, and user-kernel transitions—increases rapidly due to escalating contention for CPU resources and the overhead associated with managing a larger number of concurrent threads. This surge in software-induced delays exacerbates latency as the system scales.

In this work, we address the indexing challenges in disaggregated storage systems by proposing a DPU-oriented indexing scheme that aims to completely bypass the host CPU, thereby eliminating significant interaction and software overhead, as illustrated in Figure 1b. However, the unique characteristics of DPUs pose challenges for directly adapting existing CPU-based indexing schemes to this architecture, including the following:

Challenge #1: Limited memory resources. DPUs typically have smaller on-board memory (e.g., in the range of megabytes) compared to CPUs [3, 6, 23], while in storage servers, indexing information often demands considerable memory space [22, 67]. For example, a 1PB flash array using a 16KB page-based index structure would require 64GB of memory, far exceeding the capacity of DPU memory. As a result, existing indexing schemes must be carefully redesigned to accommodate the memory limitations of DPU devices.

Challenge #2: Weak computational power. DPUs are architected to manage a large volume of data-centric tasks in parallel, leveraging multiple cores with simpler, power-limited instructions compared to general-purpose CPU cores [39, 65].

These cores prioritize high-throughput data processing over computational complexity. Consequently, the DPU-friendly indexing scheme must be designed to minimize computational demands, aligning with the DPU’s constraints on power and instruction complexity.

Challenge #3: High interaction overhead. A DPU is typically connected to the host via the PCIe interfaces. Compared to accessing its internal memory, interacting with the host memory through PCIe leads to a significant increase in latency [43, 47, 65]. Additionally, as the number of active threads on a DPU increases, PCIe bandwidth contention worsens latency. As shown in Figure 1c, the latency of data *Transfer* can be 18 to 628 times longer than that of index lookup execution. Therefore, minimizing DPU-host interactions is critical when designing the indexing scheme.

To address these challenges, we propose a **DPU-Oriented Hybrid Indexing** scheme, **HiDPU**, tailored for disaggregated storage systems. During the index construction of HiDPU, the unified logical address space is divided into segments based on continuity. Adaptive indexing techniques are applied accordingly: linear indexing for strongly contiguous keys, a combination of linear and perfect hash indexing for weakly continuous keys, and perfect hash indexing for segments with completely random keys. A multi-layer learned index is stored on the DPU to identify the relevant segment with minimal space overhead. Due to limited memory capacity, detailed index information for each segment is stored on host memory, while essential segment metadata is cached on the DPU to reduce DPU-host interactions. Given the DPU’s limited computational power, index updates are managed by the host CPU. Host buffers real-time inserts and convert them into bulk inserts, with HiDPU incrementally updated in the background through local reconstruction. HiDPU also periodically performs global index reconstruction to mitigate imbalances caused by local updates.

We have implemented a fully functional prototype of HiDPU based on Huawei’s DPU Hi1823 to demonstrate the effectiveness of the proposed techniques. A host-based functional prototype of HiDPU is publicly accessible [54]. HiDPU can host a 1.2 PB storage system with a minimum of 23 KB of DPU memory usage. Evaluation results based on publicly available block I/O datasets and various real-world benchmark tests demonstrate that our solution reduces memory usage by up to 92% and enhances lookup efficiency by up to 6.3x compared to traditional indexing methods.

The remainder of this paper is organized as follows: Section 2 provides the background. Section 3 and Section 4 detail the design and implementation techniques, respectively. Section 5 presents the evaluation results, while Section 6 discusses related work. Finally, Section 7 concludes the paper.

2 Background

2.1 Data Processing Units

A Data Processing Unit (DPU) is a specialized processor designed to handle data-centric tasks, offloading workloads from the CPU and accelerating system data processing performance. DPUs are commonly deployed in applications such as cloud computing, disaggregated storage systems, and networking, where efficient data movement and management are crucial [28]. Numerous companies have introduced DPU products, including AMD Pensando [3], Broadcom Stingray [5], Intel IPU [23], NVIDIA BlueField [44], Zynq MPSoCs [4], Huawei Hi1823, and others [10, 24, 27, 41, 42].

As shown in Figure 2, a DPU typically comprises several key functional components: network traffic interfaces, programmable SoC cores, shared on-board memory, PCIe interfaces, and hardware accelerators. The network traffic interfaces, such as high-speed Ethernet or InfiniBand, enable the DPU to handle large volumes of network traffic in parallel (up to 100 Gbps or higher). The SoC cores are optimized for parallel data processing tasks, including custom packet processing, virtualization infrastructure, and resource management. These cores are lighter and more specialized than general-purpose CPU cores, prioritizing throughput and power efficiency. The shared on-board memory is used for caching frequently accessed data, with much lower access latency compared to host memory. However, this memory is usually limited in size (e.g., megabytes). The PCIe interfaces connect the DPU to host system via the PCIe bus, allowing high-speed data exchange between the DPU and host memory via DMA and enabling direct access to other PCIe peer devices, such as flash arrays or NVMe drives. Hardware accelerators typically include encryption/decryption engines, compression/decompression engines, and deep packet inspection (DPI).

2.2 Mapping Index Structures

Mapping structures are used to record translations between logical and physical address spaces, playing a crucial role in various applications. In a disaggregated storage system, efficient space allocation and management are achieved by exposing the back-end flash array space to the compute server as a unified sequential logical address space. The CPU then translates this logical address space into specific physical addresses on individual flash devices (as illustrated in the

Figure 1). This section provides an overview of several commonly used index structures and recent works.

Page Table. Page table-based indexing, which maintains a one-to-one mapping for each page-based logical address, is widely used [18, 26]. However, pre-allocating mapping entries for all logical pages leads to significant space inefficiency due to unused 'holes' in the logical space. To mitigate this, multi-level page table indexing structures have been proposed. As illustrated in Figure 3, a three-level page table divides the logical address space into equal-sized segments, with the leaf nodes storing actual mapping entries and the root and intermediate nodes containing pointers to the next level. This hierarchical design enables on-demand allocation of sub-tables, reducing wasted space but increasing the number of address translation lookups. Additionally, 'holes' still exist in the sub-tables at each level, leading to space overhead.

Learned Table. Learning-based indexing has gained significant attention from researchers due to its space efficiency [16, 31]. As illustrated in Figure 3, all the mapping entries are organized in the table array, ordered by their logical addresses, with learned models trained on them. Each model functions as a linear equation, and the predicted location of a target logical address (la) is determined using the model's parameters: the first key (x_0), slope (k), and intercept (b). Although the Learned Table does not contain 'holes', the model's predictions can be imprecise, necessitating a local search around the predicted location to identify the correct entry by comparing the stored logical address with the target address.

During training, the learned index adjusts error bounds ϵ to balance accuracy and the number of models. The error bound indicates the maximum number of neighboring entries visited during a local search; reducing it narrows the search scope but increases the number of models. Multi-layer learned indexes [16] introduce additional models to create a hierarchical structure, speeding up the process of locating the target key across numerous models.

Recently, LeaFTL [59] propose to use piecewise linear functions for precise LPN-PPN mapping. However, it assumes that LPN-PPN mappings are continuous and cannot store metadata such as access control or valid bits. To address these challenges, LearnedFTL [64] retains the mapping table and uses a learned index as an efficient cache, employing a bitmap to indicate the precision of the learned index.

Perfect Hash Table. A Perfect Hash Table (PHash) [48, 49] can be implemented, using double hashing to map keys to unique positions. As shown in Figure 3, a logical address is first hashed into a bucket, then subjected to a bit-wise XOR operation with the bucket's unique *pilot* value, followed by another hash to determine the final position. The *pilot* value, unique to each bucket, serves as a distinguishing constant used to perturb the hashing process. This ensures keys are remapped to non-conflicting positions, even if multiple keys hash to the same initial bucket.

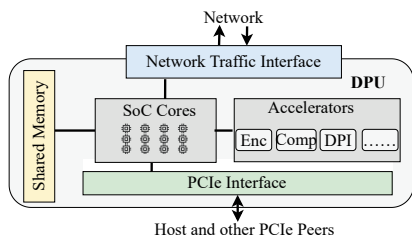


Figure 2: The general architecture of DPU.

The primary challenge in constructing a PTHash is determining an appropriate pilot value for each bucket. For each bucket, the pilot value is incremented from 0 until a value is found that ensures no key in the current bucket conflicts with keys from previous buckets. Consequently, PTHash construction is computationally intensive, with complexity depending on the number of keys and buckets involved.

3 Design of HiDPU

In this section, we present the design details of our proposed indexing scheme for disaggregated storage systems. We desire to achieve three important goals in our design.

Aim #1: Memory efficiency. The indexing structure must be memory efficient. Indexing a petabyte-scale disaggregated storage system would require gigabytes of memory, which is costly even for host memory.

Aim #2: Minimal DPU-Host interaction. Given the limited memory resources in DPUs, the indexing structure must be split between the DPU and host memory. To minimize interaction latency, the frequency of DPU-host interactions must be reduced.

Aim #3: Update efficiency and consistency. Although most workloads in disaggregated storage systems are read-intensive [32], update operations must also be handled efficiently and correctly. Additionally, after an update, the index must remain consistent between the DPU and host memory.

By following the three design goals, we propose a DPU-originated hybrid indexing structure that combines learned indexing and perfect hash table to address challenges related to memory consumption and lookup efficiency. We have developed a fully functional prototype based on Huawei’s DPU platform. Notably, the design principles outlined in this paper are applicable to not only disaggregated storage systems utilizing flash arrays but also to disaggregated memory system [62, 63] with DRAM, or other non-volatile memory (NVM) technologies. In the following sections, we will detail each design component individually.

3.1 Indexing Structure of HiDPU

As illustrated in Figure 4, the HiDPU indexing structure consists of multiple levels. The lowest level stores all the mapping entries, with each entry indexed by a logical address

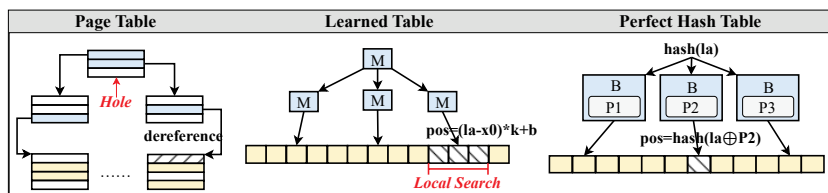


Figure 3: Different index mapping structures.

(LA) and containing its corresponding physical address (PA) along with page metadata. These mapping entries are categorized into different segments based on the continuity of their addresses: strong, weak, and random continuity. For each segment type, a distinct inner-segment indexing mechanism is built based on its mapping continuity. On top of these segments, HiDPU constructs a tiered learned table-based index (i.e., Learned Model) to enhance the efficiency of locating the segment containing the target key.

- **Accurate Segment.** For segments with strictly contiguous LAs (strong continuity), HiDPU uses linear functions to build precise and efficient segment indexes, referred to as Accurate Segments. The linear indexing parameters of an accurate segment only include a first key and an address. The *first key* determines whether a target key falls within the segment, while an *address* pointer indicates the start of the mapping array. The mapping entry for any target LA within the segment is calculated as $LA - \text{first key} + \text{address}$.

- **PTHash Segment.** For segments with a completely random distribution of LAs, HiDPU employs perfect hash tables for indexing. In addition to storing the *first key* and *address*, the indexing parameters of a PTHash segment also record the number of buckets (*bucket*) and the segment length (*len*). To locate an LA within a PTHash segment, the index first calculates $\text{hash}(LA) \% \text{bucket}$ to identify the corresponding bucket. The exact location of the mapping entry is then determined by computing $\text{hash}(LA \oplus \text{pilot}) \% \text{len}$, using the pilot value from the selected bucket.

- **LPTHash Segment.** In a PTHash Segment, while not all LAs are continuous, some may exhibit sequential patterns (weak continuity). To better utilize the internal locality within these segments, we introduce the LPTHash segment, a linear perfect hash table-based index segment. In LPTHash, the bucket of a LA is calculated using a linear function $(LA - \text{first key}) / \text{div}$, where *div* is chosen during construction to group as many LAs as possible into the same bucket. Once the bucket is selected, the exact location is determined similarly to PTHash by computing $\text{hash}(LA \oplus \text{pilot}) \% \text{len}$, using the bucket’s pilot value. The metadata of the LPTHash segment is largely identical to that of a PTHash segment, with the addition of the *div* parameter. The metadata of an LPTHash segment is largely identical to that of a PTHash segment, with the *div* parameter replacing *bucket*.

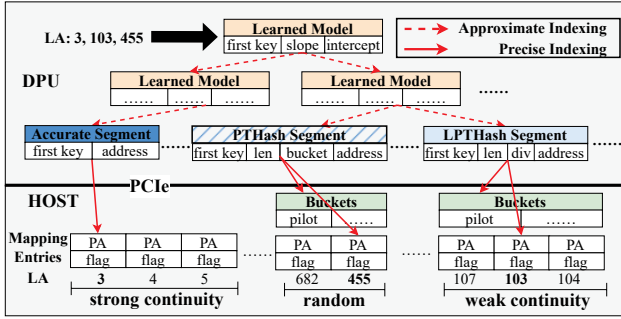


Figure 4: The hybrid indexing structure of HiDPU.

This design improves the address lookup efficiency of LPHash segments by using a linear function to group continuous LAs into the same bucket, allowing them to share the same pilot and enhancing memory access locality (as discussed in Section 5.3). Consequently, LPHash achieves a more lightweight lookup process and improved memory utilization compared to PTHash Segments, particularly in scenarios with partially continuous LAs.

However, the use of linear functions in LPHash segments presents new challenges. While PTHash utilizes a hash function to evenly distribute LAs across buckets, the linear function in LPHash can result in uneven bucket distribution, potentially generating empty buckets. Additionally, if a bucket contains too many LAs, finding a pilot value that ensures non-conflicting positions for all LAs within the bucket becomes difficult.

To address this, HiDPU imposes specific constraints on bucket number and pilot calculation during the LPHash construction process. In PTHash, the number of buckets is typically set to around $3n/\log_2(n)$ [48], where n is the number of LAs. HiDPU aims to find the minimum number of buckets between 1 and $3n/\log_2(n)$ that allows for timely construction. For a selected bucket number, the *div* is determined by the key range of current segment to distribute LAs across the buckets. The pilot search begins at 0 and increments, with an upper limit imposed; if the limit is reached, a timeout is triggered, necessitating a larger bucket count. Since there is a monotonic relationship between construction time and bucket count, a binary search can be used to quickly identify the optimal bucket number. If no suitable bucket number is found, the default PTHash is then utilized.

- **Segmentation Process.** HiDPU needs to maintain metadata for each segment in memory, so limiting the number of segments is crucial for saving memory space. However, excessively reducing the number of segment can compromise address continuity, which in turn affects indexing efficiency. To balance this trade-off, HiDPU employs an adaptive segmentation strategy. Initially, mapping entries are divided into N contiguous key segments, which are iteratively merged until only P segments remain, where P is a predefined spatial constraint.

Each segment is assigned with a merge score, s_i , and all segments are stored in a priority queue based on their score. During each iteration, the segment with the lowest score is merged with its adjacent segment. This process is repeated $N - P$ times, with the merge score updated as follows:

$$s_i = n_i + n_{i+1} + \beta \cdot d_i, \quad (1)$$

where n_i is the number of keys in a segment, d_i is the key range distance between two adjacent segments, and β adjusts the balance between segment size and key distance. This approach balances memory efficiency and address continuity, optimizing indexing performance.

After segmentation, HiDPU creates Accurate Segments for contiguous segments and LPHash/PTHash Segments for others. The algorithm retains larger segments as Accurate Segments, limiting the number of elements in the perfect hash table. Additionally, HiDPU sets a threshold for search pilots. Once exceeded, HiDPU retries with the segment length (*len*) multiplied by an amplification factor to reduce construction overhead.

Learned inner index. In the upper layer, a multi-layer learned table based index is used to locate the segments. This index generates a series of tiered models, each represented by a linear function defined by parameters such as the *first key*, *slope*, and *intercept*. When a LA is received, these models predict the segment containing the address using these functions. Unlike the exact indexing within each segment, learned indexing is approximate, potentially requiring checks of neighboring models or segments to identify the correct one.

Learned inner index is trained using the first key of each segment, with different error bound ϵ balancing space and query efficiency. Since the index is much smaller than the segment metadata, training prioritizes query efficiency. While a smaller ϵ reduces errors, it might increase the number layers. A logistic model could be used to fit the relationship between the number of layers and ϵ , incorporating DPU-measured computation and local search latency to determine the optimal ϵ value.

3.2 DPU-oriented Index Offloading

Due to the limited memory resources of the DPU, only a subset of indexes can be offloaded. Carefully selecting which indexes to offload is essential for minimizing DPU-host interactions and optimizing overall system performance.

Figure 4 illustrates how HiDPU manages indexes across the DPU and the host memory. HiDPU stores the lowest-level mapping entries and segment bucket data (i.e., the bucket pilots for PTHash and LPHash) in the host memory, while the learned models and segment indexing parameters including the first key and address of each segment and additional hash table metadata for PTHash/LPHash segments are offloaded to the DPU side.

If the LA belongs to an Accurate Segment, the DPU can directly compute the precise mapping location using the

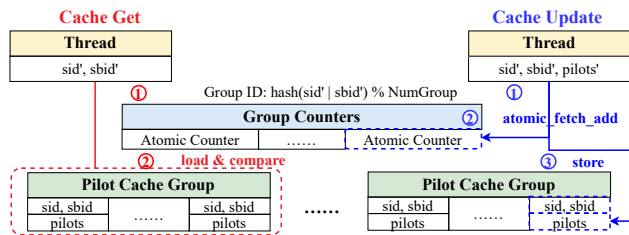


Figure 5: Working flow of Pilot Cache in HiDPU.

segment's parameters. With a single interaction with the host memory, the DPU retrieves the mapping entry. In contrast, if the LA belongs to a PTHash Segment or an LPTHash Segment, the DPU first identifies the bucket number using a hash or linear function and retrieves the corresponding pilot from the host memory. Using the pilot, the DPU calculates the exact mapping location, requiring a second access to the host memory. This process results in two interactions with the host.

To minimize interactions with the host, HiDPU incorporates a lightweight Pilot Cache within the DPU to store frequently accessed pilot values from both the PTHash and LPTHash segments. In the event of a conflict, the Pilot Cache prioritizes retaining the pilot value from the LPTHash segment. This prioritization is based on the fact that consecutive keys in the LPTHash segment are mapped to the same bucket and share a pilot value. Given that consecutive keys typically exhibit higher access locality, caching pilot values from the LPTHash segment is more advantageous.

Ensuring cache correctness becomes a significant challenge when multiple threads in the DPU concurrently execute queries and access the cache. HiDPU addresses this by utilizing a hash-based optimistic Pilot Cache, which stores both the key and its corresponding pilot value. Upon fetching from the cache, both the key and pilot value are loaded into the thread's local storage, and the pilot value is used only if the fetched key matches the queried key. The hashing algorithm is designed to minimize the likelihood of conflicts.

However, this approach results in sub-optimal cache space utilization. During the construction of a perfect hash table, pilot values typically remain small due to the enforced timeout limit. In our implementation, 16 bits are sufficient to represent all pilot values. To address this, HiDPU consolidates adjacent buckets and their pilot values into a "super-bucket." When a cache miss occurs, pilots from the super-bucket are loaded into the DPU in a single PCIe request, enhancing efficiency.

When several DPU threads attempt to update the cache simultaneously, ensuring atomicity in cache writes is critical to prevent cache inconsistency. To address this, HiDPU introduces a grouped Pilot Cache, with an atomic counter assigned to each group. As shown in Figure 5, during cache access, each thread first determines the group ID by combining and hashing the segment identifier (*sid*) with a super-bucket identifier (*sbid*) ①①. The thread then loads all data from the corresponding group into its local memory

space and compares the identifiers to locate the target pilot ②. If the target pilot is found, it can be used directly; if not, the thread retrieves the latest pilot value from the host memory. Before updating the cache, the thread acquires a unique value from the group's atomic counter and increments it by one ②. This value is used to determine the next writable position within the group, ensuring that no two threads update the same cache unit simultaneously. After that, the thread issues the memory store operation to update the pilot value ③. While this approach increases the amount of data that needs to be read, it remains efficient since accessing the DPU on-board internal memory is much faster than interacting with the host.

3.3 Asynchronous Index Updates

The power-limited cores in the DPU prevent it from handling the complex computations required for index updates. Consequently, in HiDPU, index updates are carried out on the host side to ensure efficient processing.

Updates and deletions to mapping entries can be handled through in-place modifications without impacting the index structure, whereas insertions require changes to the index. A cuckoo hash-based index manages these insertions by calculating two potential positions in the buffer via cuckoo hashing. New entries are placed in one of these positions, and if a conflict occurs, the existing key is "kicked out" to the other position, repeating until the conflict is resolved. This approach allows the DPU thread to efficiently retrieve the target key from the buffer with two hash calculations and parallel host memory accesses.

Local Reconstruction. When the buffer reaches capacity or eviction recursions during cuckoo insertion exceed a predefined threshold, the buffered data are merged into HiDPU through local reconstruction. HiDPU divides the mapped entries in the buffer into segments based on their respective ranges, enabling autonomous reconstruction within each segment. This allows multiple threads to reconstruct segments simultaneously, while the unaffected segments continue to handle queries.

Each accurate segment contains strictly contiguous logical addresses without any insertions. Therefore, insertions can only occur within PTHash/LPTHash segments and the uncovered ranges. Insertions in uncovered ranges may create new segments and require updating the learned inner index. To address this, HiDPU extends the first key field of a PTHash or LPTHash segment to the end of the previous segment. Additionally, HiDPU introduces two special perfect hash table-based segments, where address ranges are defined as $[0, \text{min-key} - 1]$ and $[\text{max-key} + 1, \infty)$, with *min-key* and *max-key* representing the global minimum and maximum keys from the last global reconstruction. This strategy partitions the entire logical address space, ensuring that any new insertions fall into one of the current PTHash-based segments.

The local reconstruction process for a PTHash/LPTHash-based segment is illustrated in Figure 6. A

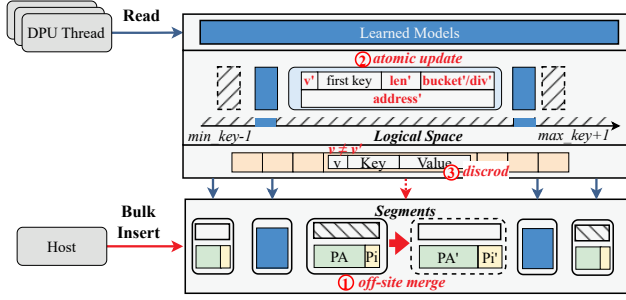


Figure 6: Illustration of HiDPU local reconstruction.

version field has been added to both the segment's metadata and the pilot cache to track whether the cached pilot has expired. Local reconstruction starts by merging new mapping table entries with copied entries from the segment's table array, constructing a new LPHash or PTHash segment, and generating new pilot values^①. During this process, the old table array continues to handle query requests from the DPU. Once the segment index is built, updated segment metadata is offloaded to the DPU, containing an incremented *version'* and an updated *address'* that points to the new table array^②. After that, the expired cache can be identified since its version *v* does not match with the up-to-date version *v'*^③. To ensure stability, the old table and pilot array are retained temporarily, allowing any threads that accessed the metadata before the update to complete their operations before the old table array is recycled.

Global Reconstruction. As the system operates, such as deletions and insertions, can alter the distribution and continuity of keys. HiDPU periodically performs a global reconstruction during idle times to adjust and reconstruct the index structure. During this process, HiDPU temporarily blocks new queries by modifying the root model of the learned indexes on the DPU and waits for a period of time until all active queries are completed. Once the pending queries are finished, HiDPU reconstructs the index structure from the bottom up, following the adaptive strategy described in Section 3.1, and re-offloads the updated structure to the DPU. Additionally, the existing cache within the DPU is cleared to ensure consistency with the newly reconstructed index.

4 Implementation

This section outlines the DPU platform used in our implementation and details how computational efficiency is further optimized through a combination of hardware and software techniques.

Platform. In this paper, we use the Huawei Hi1823 DPU as our platform for development and experimentation. The Hi1823 features hundreds of lightweight processor cores, each supporting four physical threads capable of independently processing tasks. Each thread is equipped with dedicated local memory (4KB) for frequently accessed data, while all threads share a global memory (SM, 4MB). Threads utilize

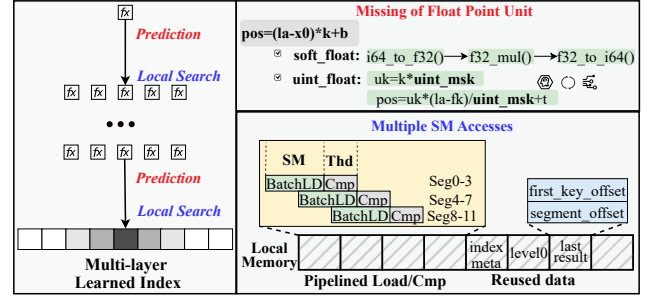


Figure 7: The DPU-Accelerated learned index processing.

DMA to transfer data between the host and local memory. As the thread count increases, SM latency ranges from 0.1 to 0.8 μ s, while DMA latency ranges from 0.6 to 17 μ s.

Note that HiDPU is designed to be compatible with classic DPU architectures, as discussed in Section 2.1, such as NVIDIA BlueField [44]. The key difference between Hi1823 and BlueField lies in their resources: Hi1823 features more lightweight cores and megabytes of memory, while BlueField offers fewer but more powerful cores, with up to 16GB of memory, which is sufficient to run HiDPU.

Fixed-Point Model Prediction. The lightweight heterogeneous cores of Hi1823, limited by power-limited instructions, lack dedicated floating-point computation units. Although the floating-point library (*soft_float*), which adheres to the IEEE 754 standard, can simulate floating-point operations, it is computationally expensive. HiDPU introduces an alternative method, *uint_float*, as depicted in Figure 7. This method transforms the slope (a floating-point number less than 1) into an integer by multiplying it with a large constant (*uint_mask*) and then dividing the result by *uint_mask* after the calculation. To prevent overflow during multiplication, the data format is temporarily expanded to 128 bits. However, there is still a margin of error between the *uint_float* method and exact floating-point calculations performed on the host during model training, which can introduce bias during inference. To mitigate this, HiDPU applies the *uint_float* method during model training, ensuring that the target key can still be located within an acceptable error range.

Pipeline-based Local Search . Lookup of learned inner index involves multiple accesses to the first key of each model or segment index, followed by comparisons with the target *LA* to determine the exact position. To improve query efficiency, we propose a pipeline-based local search that optimizes resource utilization across physical threads. Specifically, the first key of each model and segment index is stored adjacently, allowing multiple neighboring accesses to be merged, thereby reducing the number of SM accesses (referred to as *BatchLD*). Additionally, since DPU threads operate asynchronously with SM accesses, the next *BatchLD* can be initiated before the current batch's search is completed, minimizing idle time. As shown in Figure 7, key comparison (*Cmp*) and *BatchLD*

execution can occur in parallel, significantly reducing the total time required to process the learned index in the DPU.

Data Reuse. We utilize reusable data stored in local memory to minimize the frequency of SM accesses, as depicted in the bottom right corner of Figure 7. Specifically, we cache key metadata from HiDPU, such as the number of layers in the learned index, the number of models at each layer (*index meta*), and the root model parameters (*level0*), within local memory. When a thread initiates a new query, it first checks these cached locations and leverages the stored data when available. Moreover, temporary data loaded into local memory during previous queries is retained and reused for subsequent queries. The insight is that multiple consecutive requests may fall within the same segment. When these requests are scheduled to run on the same thread, the previously saved segment index data can be reused.

5 Evaluation

5.1 Experimental Setup

Our experiments were conducted on a server equipped with a 96-core 2.4GHz Intel Xeon Platinum 8260 CPU, 512GB of DDR4 DRAM, and a Huawei Hi1823 DPU, running EulerOS with Linux Kernel 4.18.0. For the default configuration, we set the β value in Eq. 1 to 1 and P to 1000, ensuring equal priority for segment lengths and distances during segment merging, with the total number of segments capped at 1000. In this setup, the HiDPU’s internal index occupies only 23.74KB of shared memory on the DPU. Additionally, the Pilot Cache size is set to 1MB by default.

In our experiments, data is structured into 4KB pages, with each page’s logical address represented by 8 bytes. The associated mapping entries, which include physical addresses and necessary page metadata, total 20 bytes. In addition to HiDPU, we developed two baseline index schemes on the Hi1823 DPU for comparison.

- **D-Page.** The first system, DPU-originated Page Table (denoted as D-Page), utilizes a three-level page table structure. The first two levels store pointers to subsequent levels, with each sub-table containing 2^{17} pointers, totaling 1MB in size. The final level stores the actual mapping entries, with 2^{18} entries per sub-table, each sub-table sized at 5MB. Due to memory constraints, only the first level of the D-Page is offloaded to the DPU.

- **D-Learned.** The second system, DPU-originated Learned Table (denoted as D-Learned), leverages multi-level learned indexes, offloading their processing to the DPU. D-Learned employs the same techniques as HiDPU to accelerate learned index searches on DPUs (as detailed in Section 4). Due to prediction errors, D-Learned needs to load additional mapping entries and compare logical addresses. To reduce errors, D-Learned generates as many learned models as possible during the build phase, while ensures that the total size does not exceed 1MB.

Table 1: Trace Configuration

Entries	wdev	web	prn	proj	oltp	websrv	tpcds	yesbc
Unique (M)	0.1	17	21	180	20	4	15	23
Total (M)	2	69	55	202	47	48	203	346

Workloads. For our workload evaluation, we assessed the performance and efficiency of D-Page, D-Learned, and HiDPU using four block I/O traces from an enterprise server, sourced from Microsoft Research Cambridge [57]. To evaluate performance under real-world applications where file systems generate sequences of consecutive logical addresses, we created a simulated SSD storage pool using FEMU [33] and captured mapping entries and access patterns during various application executions. We then employed the Filebench file system benchmarking tool [51], the Yahoo Cloud Services Benchmark (YCSB) [11] with LevelDB, and the TPC-DS Benchmark [60] with the PostgreSQL relational database to simulate diverse application scenarios and user loads. The number of unique pages and total accessed pages for all traces are shown in Table 1.

5.2 Overall Performance

In this subsection, we compare the performance of the three implementations in terms of throughput, latency and memory consumption.

5.2.1 Throughput and Latency

Micro I/O traces. To evaluate the efficiency of our proposed indexing scheme, we varied the number of concurrent clients from 1 to 256, measuring system throughput and average lookup latency across different micro I/O traces. As shown in Figure 8a, HiDPU consistently outperforms D-Page and D-Learned in both throughput and latency across all workloads. This improvement is due to HiDPU’s introduction of a continuity-based segmentation layer and the use of accurate segments and perfect hash tables, which effectively balance memory usage and query efficiency. When combined with the Pilot Cache, HiDPU achieves a 95% probability of requiring only one DMA for mapping entry retrieval. Across various workloads, HiDPU shows throughput gains of $1.1\times$ to $5.5\times$ over D-Page, and $1.4\times$ to $6.3\times$ over D-Learned. Furthermore, even with up to 256 concurrent client requests, HiDPU consistently maintains latency below $6.47\ \mu\text{s}$ across all workloads, significantly outperforming D-Page and D-Learned, which experience latencies of $22.99\ \mu\text{s}$ and $41.03\ \mu\text{s}$, respectively.

D-Page’s suboptimal performance is primarily due to its inefficient use of the DPU’s limited memory, resulting in multiple interactions with the host to access lower-level tables. As the number of client requests increases, the overhead from these interactions becomes a significant performance

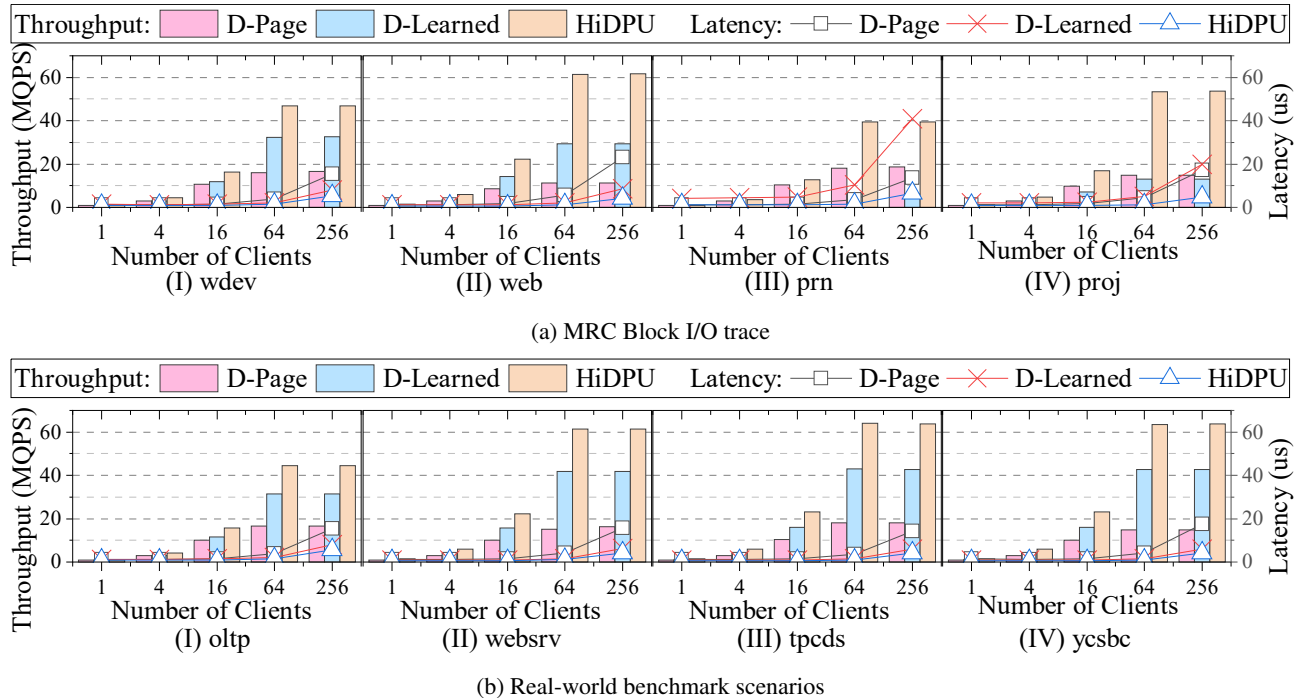


Figure 8: Overall performance of DP-Page, D-Learned, and HiDPU under (a) MRC micro block trace and (b) real-world application trace.

bottleneck. In contrast, D-Learned offloads learned indexes to the DPU, improving memory efficiency and reducing host interactions to just one when the prediction is correct, leading to throughput gains of up to $3.5\times$ over D-Page (under workload *web*). However, the effectiveness of D-Learned heavily relies on the continuity of logical addresses. When continuity is low, prediction errors increase, necessitating more DPU-host interactions.

Under the *prn*, which exhibit low continuity, D-Learned experiences up to $3.84\times$ higher latency compared to D-Page. For the *proj* workload, although the logical address space is relatively sequential, the larger data size requires D-Learned to reduce the accuracy of its learned model indexes to fit within the DPU's memory. This compromises performance, resulting in more DPU-host interactions. Consequently, the throughput of D-Learned can be up to $3.3\times$ and $1.6\times$ lower than that of D-Page, respectively.

Macro Benchmarks. We further evaluated the performance of these three implementations by replaying the macro workloads collected from the up-mentioned benchmarks. The results are given in Figure 8b. Consistently, HiDPU outperforms the other systems across all benchmarks. Overall, HiDPU achieves performance gains of $1.3\times$ to $4.3\times$ compared to D-Page, and $1.3\times$ to $1.5\times$ compared to D-Learned. In terms of latency, HiDPU maintains values below $5.76\ \mu\text{s}$ across all benchmarks, significantly outperforming D-Page and D-Learned, which show latencies of $17.39\ \mu\text{s}$ and $8.15\ \mu\text{s}$, respectively.

The throughput and latency of D-Page follow a trend similar to that observed in the micro block I/O traces, with

minor improvements. In contrast, D-Learned demonstrates better performance than in the micro traces, delivering more consistent and stable results. This improvement can be attributed to the file system's orderly allocation of logical addresses, which reduces issues related to address discontinuities. Across all benchmarks, both HiDPU and D-Learned exhibit comparatively lower performance in Filebench-based *oltp* workloads. This is mainly because *oltp* workloads typically involve small transaction sizes and random address patterns, limiting the effectiveness of the Pilot Cache and the data reuse mechanisms.

5.2.2 Memory Consumption

In disaggregated storage systems, indexing entries are typically stored in DRAM-based memory to ensure system efficiency. Maintaining mapping entries for possible logical address in a large address space with traditional page tables is memory-intensive. The "OPT" approach, which allocates entries only for used addresses, is memory-efficient but incurs high maintenance and query overhead. D-Page, D-Learned, and HiDPU offer trade-offs between these two extremes. In this subsection, we evaluate the memory consumption of the three implementations, with the comparison results normalized to "OPT" presented in Figure 9.

As shown in Figure 9a, HiDPU exhibits the lowest memory amplification across nearly all micro I/O traces and macro benchmarks, except for the *proj* workload, with amplification ranging from $1.02\times$ to $1.13\times$. HiDPU's bucket-level pilot metadata, while introducing additional memory overhead, can be compressed to under 2 bytes per bucket, keeping the

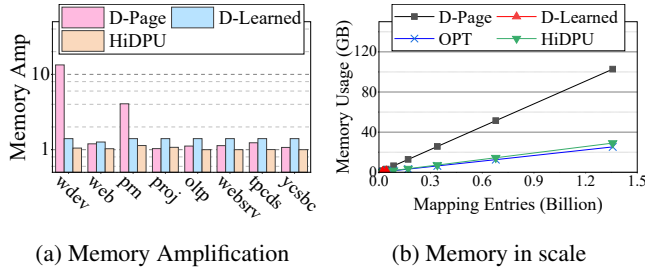


Figure 9: (a) CPU memory amplification relative to “OPT”; (b) CPU memory usage with the extended *prn* trace for billion-scale mapping entries covering petabyte-level storage.

space overhead within acceptable limits. Additionally, other metadata in HiDPU, such as segment indexes and learned indexes, incur negligible space overhead.

For D-Page, when a new mapping entry is inserted into a sub-page table, the entire sub-table must be allocated, inevitably creating ‘holes’. In the case of micro I/O traces, the randomness of logical addresses forces D-Page to allocate more sub-tables, resulting in memory amplification of up to $13.3\times$. In the *proj* workload, primarily consisting of large file accesses, the memory amplification is lower. Similarly, in the macro benchmarks, where file systems allocate addresses more continuously, memory amplification ranges from $1.06\times$ to $1.23\times$.

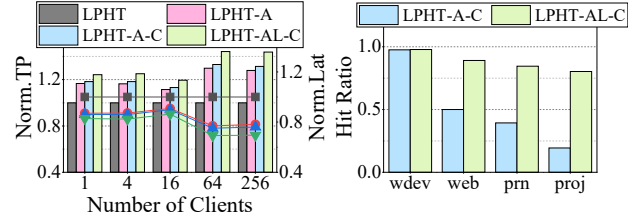
D-Learned mitigates these ‘holes’ by building prediction models on the lowest-level mapping entries. However, due to prediction inaccuracies, D-Learned must store the logical address within the mapping entries to identify the exact location, leading to a consistent memory overhead of $1.4\times$.

To assess the scalability of HiDPU, we evaluated memory requirements across varying data volumes by expanding the up-mentioned workloads. The results for the *prn* workload are shown in Figure 9b. Using a 1.26 PB of physical storage with 1 MB large pages results in 1.35 billion pages. Hence, D-Page requires 102 GB memory for the mapping table, while HiDPU only needs 29 GB, closely matching “OPT”. D-Learned struggles to balance index space and query efficiency, and fails to maintain indexes on the DPU when the page count reaches 0.04 billion. In contrast, HiDPU requires only 23 KB of DPU memory for the internal index, even with 1.35 billion pages.

5.3 Breakdown Analysis

In this subsection, we evaluate the performance improvements of each proposed module by sequentially integrating each technique as outlined below:

- **LPHT**: All mapping entries are evenly divided into P segments, with HiDPU containing only **PTHash Segments** and no pilot cache.
- **LPHT-A**: Mapping entries are divided into accurate segments and PTHash segments using a heuristic method, with no pilot cache in this configuration.



(a) Throughput and Latency. (b) Pilot Cache hit ratios.

Figure 10: (a) Performance of HiDPU variants incorporating additional techniques. (b) Pilot Cache hit ratio before (*LPHT-A-C*) and after (*LPHT-AL-C*) applying LPTHash technique.

- **LPHT-A-C**: Based on LPHT-A, a Pilot Cache is added to the DPU to buffer pilots for PTHash segment accesses.
- **LPHT-AL-C**: Incorporates all the proposed design techniques in HiDPU.

Figure 10a presents a comparison of throughput and latency across different implementations, normalized to LPHT. The introduction of accurate segments allows LPHT-A to significantly surpass LPHT in both metrics. LPHT-A improves throughput by up to $1.27\times$ and reduces latency to 78% of LPHT’s. This improvement is attributed to accurate segments, which require only a single host interaction and involve simpler index calculations. As the number of client requests increases, the performance gains of LPHT-A become more pronounced, as the cost of host interactions rises with higher client concurrency.

Compared to LPHT-A, LPHT-A-C improves average throughput by $1.03\times$ and reduces latency by 4% using the proposed pilot cache. However, the performance gains over LPHT-A are not significant due to the non-guaranteed pilot cache hit ratio. Figure 10b shows the pilot cache hit ratio across different workloads with 256 simultaneous client requests. In LPHT-A-C, the hit ratio varies significantly by workload, with a low of just 19%. This occurs because, although there is some spatial locality in logical address accesses, the hash function maps them to disparate buckets, reducing the pilot cache hit ratio.

HiDPU further leverages address continuity by converting PTHash to LPTHash, enabling continuous logical addresses to map to the same bucket. This approach improves the pilot cache hit ratio significantly across all workloads, reaching between 80% and 97%, as shown in LPHT-AL-C in Figure 10b. As a result, it reduces DPU-host interactions and achieves an additional throughput improvement of up to $1.1\times$ and reduces latency to 91% of LPHT-A-C.

5.4 Index Reconstruction

When the insertion buffer on the host side reaches capacity, local reconstruction is triggered to merge new insertion mapping entries into the leveled indexing structure of HiDPU.

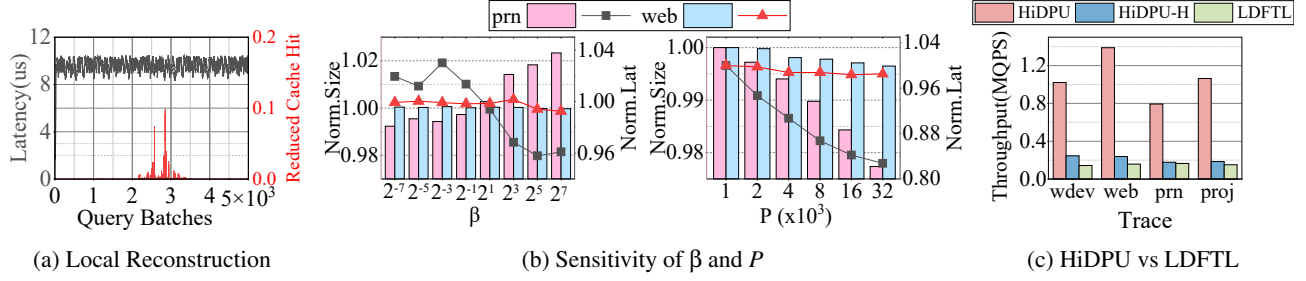


Figure 11: (a) Real-time latency and the reduced pilot cache hit ratio during local reconstruction; (b) Effects of parameter β and P on memory usage and average latency; (c) Performance comparison between HiDPU, HiDPU-Host and LearnedFTL.

In this subsection, we will evaluate the impact of local reconstruction on query performance on a mixed workload.

During the experiments, we use 256 clients to simultaneously and continuously issue requests to the DPU. After a certain time period, the host triggers local reconstruction when the number of insertions in the buffer reaches the limit. We recorded the real-time latency of one random thread and tracked the number of cached version expiration events that occurred during the experiment. Version expiration forces the current cached data to be discarded, reducing the cache hit ratio. We merge 1000 consecutive queries into a query batch and show the average latency of each query batch as well as the reduced cache hit ratio due to version expiration in Figure 11a.

The figure shows that while local reconstruction introduces some cache misses, the impact on query performance remains minimal. During local reconstruction, each PTHash and LPTHash segment is updated sequentially. If a segment is accessed after the update, its cached pilot value expires, causing cache misses and temporarily reducing the cache hit ratio by up to 9.9%. When a cache miss occurs, the query thread retrieves the latest pilot value from host memory and updates the cache. Additionally, neighboring pilot values within the same super bucket are also loaded into the DPU, enabling the cache to quickly adjust to the updated version. Overall, the average reduction in cache hit ratio during local reconstruction is only 0.15%.

The average query latency in the timeframe when local reconstruction occurs increases by only 0.69% compared to other times. This is because even when cache misses occur, techniques such as offsite reconstruction and delayed reclamation ensure that query threads can still fetch the correct data from old tables by accessing the raw data in host memory. Despite the cost of two host interactions, the pilot cache is quickly restored to the most up-to-date state.

We further evaluate the stall time for global reconstruction. The most time-consuming operation is building the perfect hash tables, which can be accelerated through concurrent multi-process execution. Therefore, with a data size of 1.2 PB, global reconstruction can be completed in under 10 minutes.

5.5 Sensitivity Analysis

The index construction of HiDPU is guided by two predefined parameters, β and P , which are designed to balance indexing bias and space requirements. In this section, we analyze the variations in HiDPU’s memory usage and query latency across different values of these parameters, based on results from two representative traces: *web* and *prn*.

During mapping table segmentation, HiDPU seeks to merge smaller segments with adjacent ones, with β controlling the trade-off between merging smaller segments or closer ones. As β increases, segments that are closer in distance are prioritized for merging. Figure 11b shows the impact of varying β . For the *web* trace, both memory usage and query latency are minimally impacted, as most mapping entries are already tightly clustered, making them less sensitive to changes in distance. In contrast, the *prn* trace shows that HiDPU’s memory overhead increases with β , reaching up to $1.02\times$ the original, while query latency decreases to 95% of the original. For non-densely clustered mapping entries, adjusting the β parameter provides a trade-off between memory consumption and query performance.

Figure 11b further illustrates the impact of the parameter P on HiDPU. Similar to the β analysis, the *web* workload is relatively insensitive to changes in P , with the proportion of the three segment index types remaining stable across different segment counts, resulting in only minor performance variations. In contrast, the *prn* workload shows significant changes as the number of segments increases. A larger exploration space enables HiDPU to identify more accurate segments and LPTHash segments, increasing the proportion of accurate segments from 8% to 10% and LPTHash segments from 72% to 84%. Consequently, query latency decreases to 82% of the baseline, while memory usage drops to 97%. Each index segment in HiDPU takes a fixed 24 bytes, and the optimal P value can be adjusted based on available DPU memory to maximize performance.

5.6 Comparison with the State-of-the-Art

In this subsection, we compare HiDPU with the state-of-the-art LearnedFTL [64]. LearnedFTL maintains

one-to-one mapping entries with the traditional page table in the SSD, while accelerating lookup with the Cached Mapping Table (CMT) and learned indexes in memory. LearnedFTL first checks the CMT; if the target entry is a miss, it then checks the learned indexes. Since the learned index can be inaccurate, a bitmap is used for each mapping entry to indicate whether the result is valid. If the result is inaccurate, LearnedFTL issues an I/O request to the page table in the SSD. In addition to the CMT and learned indexes, LearnedFTL must maintain one bit for each possible mapping entry, which makes it too large to be offloaded into DPU memory.

We compare the performance of *HiDPU*, *HiDPU* in Host (*HiDPU-H*), and LearnedFTL (*LDFTL*). While *HiDPU* and *HiDPU-H* manage to keep compacted mapping entries in memory, *LDFTL* stores mapping entries in the SSD. To ensure a fair comparison, we increased the size of the CMT and learned indexes to match *HiDPU*'s memory consumption. The experimental results for a single client are shown in Figure 11c.

HiDPU achieves the highest throughput by eliminating software overhead latency. Furthermore, *HiDPU-H* exhibits slightly higher throughput than *LDFTL* because the latter may incur I/O operations for mapping entries. Although the probability of such I/O events is reduced to as low as 0.5%, the latency associated with I/O significantly impacts performance.

6 Related Work

6.1 DPU-Assist System

DPUs are increasingly adopted in cloud data centers to offload specific computations, alleviating the computational burden on the host CPU. Related works include TAROT [58], Conspirator [66], and research across various domains such as storage systems [14, 15, 25, 36, 52], security [45, 58], cloud computing [12, 29, 35, 37, 38]. In addition, several works also propose utilizing the DPU's memory resources to cache frequently accessed data, thereby accelerating data processing performance, such as hKVS [7] and Dow-KV [70].

Further research has explored the design of DPU-oriented indexing structures that capitalize on the DPU's central position in the data path to facilitate direct data access. A notable example is Leed [17], which introduces a DPU-based Just a Bunch Of Flash (JBOF) key-value storage system. However, Leed's architecture places the external index in Flash, necessitating two NVMe accesses for each key-value request. In contrast, *HiDPU* stores as many indexes as possible within the DPU, leveraging address continuity to reduce external storage access to one in most cases.

6.2 Mapping schemes

In storage systems, space-efficient mapping tables are crucial due to the larger physical storage and significant access latency between internal and external storage. Traditional page-table-based indexing structures impose considerable memory requirements, leading to significant overhead in both system cost and performance. When memory resources are constrained, some works have proposed an on-demand approach, wherein the complete mapping table is stored externally and frequently accessed entries are cached in internal memory [8, 13, 18, 53]. Alternatively, other approaches leverage address continuity within mapping tables to achieve more efficient storage through compression, as demonstrated by [26].

Recent advancements have incorporated machine learning to enhance space with learned indexes, such as in LeaFTL [59] and LearnedFTL [64], which employ segmented linear regression to map logical to physical addresses. Building on these innovations, *HiDPU* leverages address continuity to construct separate segment indexes, thereby improving space and query efficiency. To overcome the DPU's limited memory space, *HiDPU* utilizes data locality to implement a pilot cache with on-demand access, significantly reducing interaction frequency.

7 Conclusion

In this work, we introduce *HiDPU*, a hybrid index architecture designed to facilitate index lookups on DPUs, eliminating the software overhead associated with host CPU involvement. To overcome challenges such as limited DPU memory, low computational power, and high DPU-host interaction overhead, *HiDPU* integrates learned index mapping structures with perfect hash tables and linear compression, optimizing internal index space while reducing data interactions with host memory. Additionally, we propose a DPU-based cache to enhance performance and implement a two-level index reconstruction strategy for efficient index updates. Through implementation on Huawei's Hi1823 DPU, our experimental results show significant improvements in query processing and memory efficiency compared to existing solutions.

8 Acknowledgments

The work described in this paper is partially supported by the Key R&D Program of Shandong Province under Grant 2023CXPT002, the National Natural Science Foundation of China under Grant 62272271, 624B2091, 62072333 and the Research Grants Council of the Hong Kong Special Administrative Region, China under Grant GRF 14202123, GRF 14200224.

References

- [1] Storage performance development kit. <https://spdk.io/>. Accessed: 2024-09-09.
- [2] AGRAWAL, S. R., IDICULA, S., RAGHAVAN, A., VLACHOS, E., GOVINDARAJU, V., VARADARAJAN, V., BALKESSEN, C., GIANNIKIS, G., ROTH, C., AGARWAL, N., ET AL. A many-core architecture for in-memory data processing. In *IEEE/ACM International Symposium on Microarchitecture (Micro)* (2017), pp. 245–258.
- [3] AMD. Pensando accelerators. <https://www.amd.com/en/products/accelerators/pensando.html>, 2024. Accessed: 2024-08-29.
- [4] AMD. Zynq ultrascale+ mpso. <https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-ultrascale-plus-mpso.html>, 2024. Accessed: 2024-08-29.
- [5] BROADCOM. Broadcom product release 53106. <https://www.broadcom.com/company/news/product-releases/53106>, 2024. Accessed: 2024-08-29.
- [6] BURSTEIN, I. Nvidia data center processing unit (dpu) architecture. In *IEEE Hot Chips Symposium (HCS)* (2021), IEEE, pp. 1–20.
- [7] CHEN, H.-H., CHANG, C.-H., AND HUNG, S.-H. Hkvs: a framework for designing a high throughput heterogeneous key-value store with smartnic and rdma. In *Conference on Research in Adaptive and Convergent Systems (RACS)* (2022), pp. 99–106.
- [8] CHEN, R., QIN, Z., WANG, Y., LIU, D., SHAO, Z., AND GUAN, Y. On-demand block-level address mapping in large-scale nand flash storage systems. *IEEE Transactions on Computers (TC)* 64, 6 (2014), 1729–1741.
- [9] CHEN, Y., CHEN, J., WANG, Y., CHEN, Y., JIN, Z., XU, J., FANG, G., LIN, W., WEI, C., AND CHEN, W. Hyq: Hybrid i/o queue architecture for nvme over fabrics to enable high-performance hardware offloading. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)* (2023), IEEE, pp. 13–24.
- [10] CLOUD, A. Accelerating the future with alibaba cloud. <https://www.alibabacloud.com/blog/599183>, 2024. Accessed: 2024-08-29.
- [11] COOPER, B. F. Ycsb: Yahoo! cloud serving benchmark. <https://github.com/brianfrankcooper/YCSB>. Accessed: Aug. 10, 2024.
- [12] DAW, N., BELLUR, U., AND KULKARNI, P. Speedo: Fast dispatch and orchestration of serverless workflows. In *ACM Symposium on Cloud Computing (SoCC)* (2021), pp. 585–599.
- [13] DAYAN, N., BONNET, P., AND IDREOS, S. Geckoftl: Scalable flash translation techniques for very large flash devices. In *International Conference on Management of Data (MOD)* (2016), pp. 327–342.
- [14] DING, C., ZHOU, J., LU, K., LI, S., XIONG, Y., WAN, J., AND ZHAN, L. D2comp: Efficient offload of lsm-tree compaction with data processing units on disaggregated storage. *ACM Transactions on Architecture and Code Optimization* 21, 3 (2024), 1–22.
- [15] DING, C., ZHOU, J., WAN, J., XIONG, Y., LI, S., CHEN, S., LIU, H., TANG, L., ZHAN, L., LU, K., ET AL. Dcomp: Efficient offload of lsm-tree compaction with data processing units. In *International Conference on Parallel Processing (ICPP)* (2023), pp. 233–243.
- [16] FERRAGINA, P., AND VINCIGUERRA, G. The pgm-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1162–1175.
- [17] GUO, Z., ZHANG, H., ZHAO, C., BAI, Y., SWIFT, M., AND LIU, M. Leed: A low-power, fast persistent key-value store on smartnic jbofs. In *ACM SIGCOMM Conference (SIGCOMM)* (2023), pp. 1012–1027.
- [18] GUPTA, A., KIM, Y., AND URGAKAR, B. Dftl: a flash translation layer employing demand-based selective caching of page-level address mappings. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2009), p. 229–240.
- [19] HAAS, G., HAUBENSCHILD, M., AND LEIS, V. Exploiting directly-attached nvme arrays in dbms. In *CIDR* (2020), vol. 20, p. 2.
- [20] HARRIS, B., AND ALTIPARMAK, N. {Ultra-Low} latency {SSDs}’ impact on overall energy efficiency. In *USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)* (2020).
- [21] HUAWEI. Flashlink. <https://support.huawei.com/enterprise/en/doc/EDOC1100253812/4fe20822/flashlink>, 2024. Accessed: 2024-08-26.
- [22] HUAWEI TECHNOLOGIES CO., LTD. Oceanstor dorado all-flash storage. <https://e.huawei.com/hu/products/storage/all-flash-storage>, 2024. Accessed: 2024-08-17.
- [23] INTEL. Infrastructure processing unit (ipu). <https://www.intel.com/content/www/us/en/products/details/network-io/ipu.html>, 2024. Accessed: 2024-08-29.
- [24] INTEL, AND CLOUD, G. Intel and google cloud jointly launch data center accelerator chip. <https://www.datacenterdynamics.com/en/news/intel-and-google-cloud-protect-penalty-mjointly-launch-data-center-accelerator-chip/>, 2024. Accessed: 2024-08-29.
- [25] JI, H., MANSI, M., SUN, Y., YUAN, Y., HUANG, J., KUPER, R., SWIFT, M. M., AND KIM, N. S. {STYX}: Exploiting {SmartNIC} capability to reduce datacenter memory tax. In *USENIX Annual Technical Conference (ATC)* (2023), pp. 619–633.
- [26] JIANG, S., ZHANG, L., YUAN, X., HU, H., AND CHEN, Y. S-ftl: An efficient address translation for flash memory by exploiting spatial locality. In *IEEE Symposium on Mass Storage Systems and Technologies (MSST)* (2011), pp. 1–12.
- [27] KALRAY. Mppa technology. <https://www.kalrayinc.com/products/mppa-technology>, 2024. Accessed: 2024-08-29.
- [28] KFOURY, E. F., CHOUERI, S., MAZLOUM, A., ALSABEH, A., GOMEZ, J., AND CRICHIGNO, J. A comprehensive survey on smartnics: Architectures, development models, applications, and research directions. *IEEE Access* 12 (2024), 107297–107336.
- [29] KHALILOV, M., CHRAPEK, M., SHEN, S., VEZZU, A., BENZ, T., DI GIROLAMO, S., SCHNEIDER, T., DE SENSI, D., BENINI, L., AND HOEFLER, T. {OSMOSIS}: Enabling {Multi-Tenancy} in datacenter {SmartNICs}. In *SENIX Annual Technical Conference (ATC)* (2024), pp. 247–263.
- [30] KLIMOVIC, A., KOZYRAKIS, C., THERESKA, E., JOHN, B., AND KUMAR, S. Flash storage disaggregation. In *ACM European Conference on Computer Systems (EuroSys)* (2016), pp. 1–15.
- [31] KRASKA, T., BEUTEL, A., CHI, E. H., DEAN, J., AND POLYZOTIS, N. The case for learned index structures. In *ACM International Conference on Management of Data (SIGMOD)* (2018), pp. 489–504.
- [32] LI, C., CHEN, H., ZHANG, S., HU, Y., CHEN, C., ZHANG, Z., LI, M., LI, X., HAN, D., CHEN, X., ET AL. Bytegraph: a high-performance distributed graph database in bytedance. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3306–3318.
- [33] LI, H., HAO, M., TONG, M. H., SUNDARARAMAN, S., BJØRLING, M., AND GUNAWI, H. S. The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator. In *USENIX Conference on File and Storage Technologies (FAST)* (2018), pp. 83–90.
- [34] LI, P., HUA, Y., ZUO, P., CHEN, Z., AND SHENG, J. {ROLEX}: A scalable {RDMA-oriented} learned {Key-Value} store for disaggregated memory systems. In *USENIX Conference on File and Storage Technologies (FAST)* (2023), pp. 99–114.
- [35] LI, Q., CHEN, L., WANG, X., HUANG, S., XIANG, Q., DONG, Y., YAO, W., HUANG, M., YANG, P., LIU, S., ET AL. Fisc: a large-scale cloud-native-oriented file system. In *USENIX Conference on File and Storage Technologies (FAST)* (2023), pp. 231–246.

- [36] LIAO, Y., WU, J., LU, W., LI, X., AND YAN, G. Dpu-direct: Unleashing remote accelerators via enhanced rdma for disaggregated datacenters. *IEEE Transactions on Computers* 73, 8 (2024), 2081–2095.
- [37] LIN, J., CARDOZA, A., KHAN, T., RO, Y., STEPHENS, B. E., WASSEL, H., AND AKELLA, A. {RingLeader}: efficiently offloading {Intra-Server} orchestration to {NICs}. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (2023), pp. 1293–1308.
- [38] LIU, G., ZHAO, L., LI, Y., DUAN, Z., CHEN, S., HU, Y., SU, Z., AND QU, W. Fuyao: Dpu-enabled direct data transfer for serverless computing. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems, (ASPLOS)* (2024), pp. 431–447.
- [39] LIU, M., CUI, T., SCHUH, H., KRISHNAMURTHY, A., PETER, S., AND GUPTA, K. Offloading distributed applications onto smartnics using ipipe. In *ACM SIGCOMM Conference (SIGCOMM)*. 2019, pp. 318–333.
- [40] MARCHUK, A. Spdk activities in nvidia. https://ci.spdk.io/download/2021-virtual-forum-prc/Topic_19_2021_PRC_SPDK_State_of_the_NVidia_activities_in_SPDK_project_Nvidia_Alexey.pdf, 2021. Accessed: 2024-09-09.
- [41] MARVELL. Data processing units (dpus). <https://www.marvell.com/products/data-processing-units.html>, 2024. Accessed: 2024-08-29.
- [42] MICROSOFT. Microsoft announces acquisition of fungible to accelerate datacenter innovation. <https://blogs.microsoft.com/blog/2023/01/09/microsoft-announces-acquisition-of-fungible-to-protect-penalty-@Maccelerate-datacenter-innovation/>, 2023. Accessed: 2024-09-17.
- [43] NEUGEBAUER, R., ANTICHI, G., ZAZO, J. F., AUDZEVIČH, Y., LÓPEZ-BUEDO, S., AND MOORE, A. W. Understanding pcie performance for end host networking. In *ACM SIGCOMM Conference (SIGCOMM)* (2018), pp. 327–341.
- [44] NVIDIA. Data processing unit (dpu). <https://www.nvidia.com/en-us/networking/products/data-processing-unit/>, 2024. Accessed: 2024-08-29.
- [45] PANDA, S., FENG, Y., KULKARNI, S. G., RAMAKRISHNAN, K., DUFFIELD, N., AND BHUYAN, L. N. Smartwatch: Accurate traffic analysis and flow-state tracking for intrusion prevention using smartnics. In *International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)* (2021), pp. 60–75.
- [46] PANG, X., AND WANG, J. Understanding the performance implications of the design principles in storage-disaggregated databases. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.
- [47] PENG, Y., LI, B., LIU, D., AND PENG, X. A high speed dma transaction method for pci express devices. In *IEEE Circuits and Systems International Conference on Testing and Diagnosis (ICTD)* (2009), IEEE, pp. 1–4.
- [48] PIBIRI, G. E., AND TRANI, R. Pthash: Revisiting fch minimal perfect hashing. In *ACM Conference on Research and Development in Information Retrieval (SIGIR)* (2021), p. 1339–1348.
- [49] PIBIRI, G. E., AND TRANI, R. Parallel and external-memory construction of minimal perfect hash functions with pthash. *IEEE Transactions on Knowledge and Data Engineering* 36, 3 (2024), 1249–1259.
- [50] POURHABIBI, A., SUTHERLAND, M., DAGLIS, A., AND FALSAFI, B. Cerebros: Evading the rpc tax in datacenters. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (New York, NY, USA, 2021), MICRO '21, Association for Computing Machinery, p. 407–420.
- [51] PROJECT, F. Filebench: File system workload generation. <https://github.com/filebench/filebench>. Accessed: Aug. 10, 2024.
- [52] PSISTAKIS, A., CHAIX, F., AND TORRELLAS, J. Minos: Distributed consistency and persistency protocol implementation & offloading to smartnics. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2024), pp. 1–17.
- [53] QIN, Z., WANG, Y., LIU, D., AND SHAO, Z. Demand-based block-level address mapping in large-scale nand flash storage systems. In *IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)* (2010), pp. 173–182.
- [54] QUIEOO. Impthash: A learning and perfect hashing implementation for mapping table. <https://github.com/quieoo/impthash>, 2024. Accessed: 2024-09-17.
- [55] SHU, J., ZHU, R., MA, Y., HUANG, G., MEI, H., LIU, X., AND JIN, X. Disaggregated raid storage in modern datacenters. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2023), pp. 147–163.
- [56] SHU, J., ZHU, R., MA, Y., HUANG, G., MEI, H., LIU, X., AND JIN, X. Disaggregated raid storage in modern datacenters. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2023), pp. 147–163.
- [57] SNIA. Block i/o trace: Microsoft research cambridge - msr cambridge block i/o traces. <http://iotta.snia.org/traces/block-io/388>, 2011. Accessed: 2024-09-17.
- [58] SONG, C., KIM, M. J., WANG, T., JI, H., HUANG, J., JEONG, I., PARK, J., NAM, H., WI, M., AHN, J. H., ET AL. Tarot: A cxl smartnic-based defense against multi-bit errors by row-hammer attacks. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2024), pp. 981–998.
- [59] SUN, J., LI, S., SUN, Y., SUN, C., VUCINIC, D., AND HUANG, J. Leafit: A learning-based flash translation layer for solid-state drives. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2023), pp. 442–456.
- [60] (TPC), T. P. P. C. Tpc-ds: Decision support benchmark. <https://www.tpc.org/tpcds/>. Accessed: Aug. 10, 2024.
- [61] VUPPALAPATI, M., MIRON, J., AGARWAL, R., TRUONG, D., MOTIVALA, A., AND CRUANES, T. Building an elastic query engine on disaggregated storage. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (2020), pp. 449–462.
- [62] WANG, J., LI, C., MEI, J., HE, H., WANG, T., WANG, P., ZHANG, L., GUO, M., WU, H., CHEN, D., AND LIU, X. Hyfarm: Task orchestration on hybrid far memory for high performance per bit. In *2022 IEEE 40th International Conference on Computer Design (ICCD)* (2022), pp. 33–41.
- [63] WANG, J., YANG, H., LI, C., ZHUANSUN, Y., YUAN, W., XU, C., HOU, X., GUO, M., HU, Y., AND ZHAO, Y. Boosting data center performance via intelligently managed multi-backend disaggregated memory. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis* (2024), pp. 1–18.
- [64] WANG, S., LIN, Z., WU, S., JIANG, H., ZHANG, J., AND MAO, B. Learnedftl: A learning-based page-level ftl for reducing double reads in flash-based ssds. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2024), pp. 616–629.
- [65] WEI, X., CHENG, R., YANG, Y., CHEN, R., AND CHEN, H. Characterizing off-path {SmartNIC} for accelerating distributed systems. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (2023), pp. 987–1004.
- [66] XIAO, Y., TOOTAGHAJ, D. Z., DHAKAL, A., CAO, L., SHARMA, P., AND KUZMANOVIC, A. Conspirator: {SmartNIC-Aided} control plane for distributed {ML} workloads. In *USENIX Annual Technical Conference (ATC)* (2024), pp. 767–784.

- [67] XU, J., CHEN, Y., WANG, Y., SHI, W., FANG, G., CHEN, Y., LIAO, H., WANG, Y., LIN, H., JIN, Z., ET AL. Lightpool: A nvme-of-based high-performance and lightweight storage pool architecture for cloud-native distributed database. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2024), IEEE, pp. 983–995.
- [68] YU, Q., GUO, C., ZHUANG, J., THAKKAR, V., WANG, J., AND CAO, Z. Caas-lsm: compaction-as-a-service for lsm-based key-value stores in storage disaggregated infrastructure. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.
- [69] ZHANG, Q., BERNSTEIN, P., CHANDRAMOULI, B., HU, J., AND ZHENG, Y. Dds: Dpu-optimized disaggregated storage. *arXiv preprint arXiv:2407.13618* (2024).
- [70] ZHANG, Y., LI, G., WAN, J., WANG, J., LI, J., YAO, T., WU, H., AND WANG, D. Dow-kv: A dpu-offloaded and write-optimized key-value store on disaggregated persistent memory. In *IEEE International Conference on Cluster Computing (CLUSTER)* (2023), pp. 271–283.

A Artifact Appendix

Abstract

The artifact provides the implementation source code of the HiDPU index structure: LMPHASH, a multi-level index structure combining learned indexes and perfect hash tables. Additionally, it includes a CPU-based simulator that incorporates the complete implementations of D-Page, D-Learned, and HiDPU, with all operations performed on the CPU. The state-of-the-art LearnedFTL is also included in this artifact.

Scope

The artifact provides a usable library (LMPHASH) that allows querying large-scale datasets using only a limited internal index. Additionally, the source code of this artifact includes all the techniques used in the paper and is implemented using general code running solely on the CPU, demonstrating the compatibility and portability of HiDPU. This artifact facilitates a deeper understanding of how HiDPU works and offers guidance on using the design on other DPUs.

Contents

The source code in the artifact is divided into two main parts: one is the LMPHASH index, implemented using the open-source libraries "pthash" and "PGM-Index", with a simple usage example ("clmpthash_test.c") provided in the "example" folder. The other part is the HiDPU Simulator, also located in the "example" folder, which includes the index construction, unloading, and querying processes for D-Page, D-Learned, and HiDPU, as well as the index implementation from LearnedFTL.

Hosting

The artifact is available at <https://github.com/quieoo/lmpthash>. The *master* branch has the latest content.